MunimAI - Complete Production-Grade Implementation Plan

Context

What: MunimAI (मुनीम AI) — India's first agentic AI business operating system for 63M SMBs, built on Paytm's ecosystem. Not a dashboard, not a chatbot — an autonomous digital muneem that ACTS on behalf of the merchant.

Why: India's SMBs have payment terminals but zero business intelligence. Rs 30L Cr credit gap, $500B trapped in udhari, Rs 3,000 Cr GST penalties, 82% fail from cash flow blindness. Khatabook raised $200M and failed because information tools don't work — only OUTCOME tools do.

Hackathon: Fin-O-Hack by Paytm (Fortune'26 at DTU). Track 2: AI for Small Businesses. Team HackPS (5 members).

Philosophy: Build the BEST product — no shortcuts, no "good enough." Every AI component uses state-of-the-art research-backed approaches. Full production-grade, end-to-end.

PHASE 1: Foundation Infrastructure

1.1 Project Structure (Turborepo Monorepo)

```
munim-ai/
├── turbo.json
├── package.json
├── .github/
│   └── workflows/
│       ├── ci.yml                    # Lint + test + build on every PR
│       └── deploy.yml                # Auto-deploy on main merge
├── apps/
│   ├── web/                          # Next.js 14 App Router — Merchant
Dashboard
│   │   ├── src/
│   │   │   ├── app/
│   │   │   │   ├── layout.tsx
```

```
|   |   |   |   ├── page.tsx                    # Dashboard (hero page)
|   |   |   |   ├── (dashboard)/
|   |   |   |   |   ├── udhari/page.tsx         # Full udhari management
|   |   |   |   |   ├── forecast/page.tsx       # Cash flow forecast
|   |   |   |   |   ├── gst/page.tsx            # GST autopilot
|   |   |   |   |   ├── customers/page.tsx      # Customer pulse radar
|   |   |   |   |   ├── schemes/page.tsx        # Government scheme
navigator
|   |   |   |   |   ├── inventory/page.tsx      # Smart inventory
inference
|   |   |   |   |   ├── employees/page.tsx      # Employee payment tool
|   |   |   |   |   └── settings/page.tsx       # Merchant profile +
preferences
|   |   |   |   ├── whatsapp/page.tsx           # WhatsApp conversation
view
|   |   |   |   ├── soundbox/page.tsx           # Soundbox simulator +
demo
|   |   |   |   └── onboarding/page.tsx         # First-time setup flow
|   |   |   ├── components/
|   |   |   |   ├── dashboard/
|   |   |   |   |   ├── LivePnLCard.tsx         # Hero card —
income/expense/profit with live animation
|   |   |   |   |   ├── UdhariTracker.tsx       # Debtor list with
remind/collect buttons
|   |   |   |   |   ├── PayScoreGauge.tsx       # Circular SVG gauge 0-100
with gradient
|   |   |   |   |   ├── CustomerPulse.tsx       # Churn risk mini-cards
with RFM
|   |   |   |   |   ├── CashFlowMini.tsx        # Sparkline 30-day
forecast
|   |   |   |   |   ├── GSTStatusBadge.tsx      # Filing status pill with
countdown
|   |   |   |   |   ├── ActivityFeed.tsx        # Real-time event log
(scrolling)
|   |   |   |   |   ├── AlertBanner.tsx         # Slide-in cash crunch /
negative profit alert
|   |   |   |   |   ├── SchemeMatch.tsx         # Government scheme
recommendation cards
|   |   |   |   |   └── InsuranceRecommender.tsx # Contextual micro-
insurance cards
|   |   |   |   ├── voice/
```

```
|   |   |   |   |   ├── VoiceInput.tsx          # Mic button + recording
state machine
|   |   |   |   |   ├── VoiceWaveform.tsx       # Audio visualizer
(canvas-based)
|   |   |   |   |   ├── SoundboxSimulator.tsx   # Visual Soundbox with LED
+ speaker output
|   |   |   |   |   └── TranscriptDisplay.tsx   # Shows transcript +
intent + entities
|   |   |   |   ├── whatsapp/
|   |   |   |   |   ├── ChatWindow.tsx          # WhatsApp-clone
conversation UI
|   |   |   |   |   ├── MessageBubble.tsx       # Text + voice + payment
link bubbles
|   |   |   |   |   ├── MorningBriefing.tsx     # Formatted briefing
message
|   |   |   |   |   └── QuickReplyButtons.tsx   # "Haan, bhej do" / "Baad
mein" buttons
|   |   |   |   ├── charts/
|   |   |   |   |   ├── CashFlowChart.tsx       # Line chart with
confidence bands + festival markers
|   |   |   |   |   ├── ProfitMarginBar.tsx     # Animated progress with
color thresholds
|   |   |   |   |   ├── ExpenseDonut.tsx        # Category breakdown
donut/pie
|   |   |   |   |   ├── RevenueHeatmap.tsx      # Day-of-week x hour
heatmap
|   |   |   |   |   ├── PayScoreHistory.tsx     # Score trend line over
months
|   |   |   |   |   └── CustomerRFMScatter.tsx  # RFM 3D scatter plot
|   |   |   |   ├── common/
|   |   |   |   |   ├── AnimatedNumber.tsx      # Framer Motion spring-
based counter
|   |   |   |   |   ├── Navbar.tsx              # Top nav with shop name +
PayScore badge
|   |   |   |   |   ├── BottomNav.tsx           # Mobile bottom tab
navigation
|   |   |   |   |   ├── LiveDot.tsx             # Pulsing green "LIVE"
indicator
|   |   |   |   |   └── HindiLabel.tsx          # Bilingual label
component
|   |   |   |   └── onboarding/
```

```
|   |   |   |       ├── LanguageSelect.tsx      # Voice-first language
selection
|   |   |   |       ├── BusinessProfile.tsx     # Shop type, location,
employees
|   |   |   |       └── UdhariImport.tsx        # Photo upload for paper
ledger OCR
|   |   |   ├── hooks/
|   |   |   |   ├── useSocket.ts               # Socket.IO connection +
auto-reconnect
|   |   |   |   ├── useVoiceRecorder.ts        # MediaRecorder + audio
blob handling
|   |   |   |   ├── useDashboardState.ts       # Central reactive state
from WebSocket
|   |   |   |   ├── useAnimatedValue.ts        # Framer Motion spring
wrapper
|   |   |   |   └── useSoundEffects.ts         # Audio playback for
income/expense
|   |   |   ├── lib/
|   |   |   |   ├── supabase.ts                # Supabase client singleton
|   |   |   |   ├── socket.ts                  # Socket.IO client
singleton
|   |   |   |   ├── api.ts                     # Typed API client (fetch
wrapper)
|   |   |   |   ├── formatters.ts              # Indian number formatting
(lakhs/crores)
|   |   |   |   └── constants.ts               # Colors, thresholds,
festival dates
|   |   |   └── types/
|   |   |       └── index.ts                   # All TypeScript interfaces
|   |   ├── public/
|   |   |   ├── sounds/
|   |   |   |   ├── ka-ching.mp3               # Income received sound
|   |   |   |   ├── expense-whoosh.mp3         # Expense logged sound
|   |   |   |   ├── alert-chime.mp3            # Alert/warning sound
|   |   |   |   └── collection-success.mp3     # Udhari collected
celebration
|   |   |   └── images/
|   |   |       ├── munim-logo.svg
|   |   |       ├── soundbox-illustration.svg
|   |   |       └── paytm-logo.svg
|   |   ├── tailwind.config.ts
```

```
|   |   └── next.config.js
|   |
|   └── whatsapp-bot/                 # WhatsApp Business API handler
|       ├── src/
|       |   ├── index.ts              # Express server for webhook
|       |   ├── webhook.ts            # Meta Cloud API webhook handler
|       |   ├── messageHandler.ts     # Route incoming messages to NLU
|       |   ├── templates.ts          # Pre-approved WhatsApp message
templates
|       |   └── mediaHandler.ts       # Voice note + image processing
|       └── package.json
|
├── services/
|   ├── ai-engine/                    # FastAPI — Core AI/ML orchestration
|   |   ├── main.py                   # FastAPI app + Socket.IO mount +
CORS
|   |   ├── config.py                 # Pydantic Settings from env
|   |   ├── routers/
|   |   |   ├── voice.py              # POST /api/voice/process (audio ->
NLU -> action)
|   |   |   ├── transactions.py       # CRUD for income/expense entries
|   |   |   ├── udhari.py             # CRUD + collection trigger +
settlement
|   |   |   ├── dashboard.py          # GET aggregated P&L, stats, alerts
|   |   |   ├── forecast.py           # GET cash flow predictions
|   |   |   ├── payscore.py           # GET/POST credit score endpoints
|   |   |   ├── gst.py                # GET status, POST auto-classify,
POST file
|   |   |   ├── customers.py          # GET customer profiles, churn risk
|   |   |   ├── schemes.py            # GET matched government schemes
|   |   |   ├── inventory.py          # GET stock inference + reorder
alerts
|   |   |   ├── employees.py          # CRUD employee + salary management
|   |   |   ├── whatsapp.py           # Send/receive WhatsApp messages
|   |   |   ├── briefing.py           # Generate morning briefing
|   |   |   └── demo.py               # Reset, simulate payment, trigger
events
|   |   ├── services/
|   |   |   ├── nlu/
|   |   |   |   ├── pipeline.py       # Full NLU orchestration pipeline
|   |   |   |   ├── stt_engine.py     # IndicWhisper / Groq Whisper STT
```

```
|   |   |   |   ├── intent_classifier.py  # IndicBERTv2 fine-tuned
classifier
|   |   |   |   ├── entity_extractor.py   # spaCy NER + Hindi numeral
parser
|   |   |   |   ├── hindi_numerals.py     # Custom Hindi numeral -> int
converter
|   |   |   |   └── code_switch_handler.py # HingBERT code-switching
detection
|   |   |   ├── agents/
|   |   |   |   ├── master_agent.py       # LangGraph orchestrator with
Muneem personality
|   |   |   |   ├── collection_agent.py   # RL-based udhari collection
(Thompson Sampling -> DQN)
|   |   |   |   ├── cashflow_agent.py     # TFT + Chronos ensemble
forecasting
|   |   |   |   ├── payscore_agent.py     # TabNet + HAN 47-feature credit
engine
|   |   |   |   ├── gst_agent.py          # IndicBERTv2 HSN classifier +
filing
|   |   |   |   ├── customer_agent.py     # TS2Vec churn detection +
winback
|   |   |   |   ├── scheme_agent.py       # CRAG + BGE-M3 cross-lingual
RAG
|   |   |   |   └── inventory_agent.py    # Transaction clustering +
demand forecast
|   |   |   ├── ml/
|   |   |   |   ├── tft_forecaster.py     # Temporal Fusion Transformer
(Google, 2021)
|   |   |   |   ├── chronos_forecaster.py # Amazon Chronos foundation
model
|   |   |   |   ├── tabnet_scorer.py      # TabNet (Arik & Pfister, 2021)
for PayScore
|   |   |   |   ├── han_graph.py          # Hierarchical Attention Network
on account data
|   |   |   |   ├── ts2vec_embedder.py    # TS2Vec (Yue et al., AAAI 2022)
contrastive learning
|   |   |   |   ├── thompson_sampler.py   # Thompson Sampling for
collection bandit
|   |   |   |   ├── dqn_collector.py      # Deep Q-Network for advanced
collection RL
|   |   |   |   └── ensemble.py           # Model ensemble + weighted
```

```
averaging
|   |   |   ├── tts_service.py            # Sarvam Bulbul Hindi TTS
|   |   |   ├── ocr_service.py            # PaddleOCR for paper udhari
photos
|   |   |   ├── action_router.py          # Intent -> DB write ->
WebSocket emit
|   |   |   └── realtime.py               # Socket.IO event manager
|   |   ├── models/
|   |   |   ├── schemas.py                # Pydantic request/response
models
|   |   |   └── db.py                     # Supabase async client + query
helpers
|   |   ├── data/
|   |   |   ├── seed_merchant.py          # Sunita Saree Shop full 6-month
dataset
|   |   |   ├── intent_training_data.json # 500+ labeled Hindi intent
examples
|   |   |   ├── hindi_numerals_map.json   # 40+ spoken Hindi numeral
patterns
|   |   |   ├── hsn_codes.json            # 18,000+ HSN/SAC codes for GST
|   |   |   ├── festival_calendar.json    # 30+ Indian festivals with
dates
|   |   |   ├── government_schemes.json   # 50+ MSME schemes with
eligibility criteria
|   |   |   └── collection_templates.json # Culturally-aware Hindi message
templates
|   |   ├── requirements.txt
|   |   └── Dockerfile
|   |
|   └── ml-training/                  # Offline ML training pipelines
|       ├── train_intent_classifier.py    # Fine-tune IndicBERTv2 on
custom intents
|       ├── train_tft_forecaster.py       # Train TFT on synthetic
merchant data
|       ├── train_tabnet_payscore.py      # Train TabNet for credit
scoring
|       ├── train_ts2vec_churn.py         # TS2Vec contrastive pre-
training for churn
|       ├── train_hsn_classifier.py       # Fine-tune IndicBERTv2 for HSN
code classification
|       ├── generate_synthetic_data.py    # Full synthetic Paytm merchant
```

```
dataset generator
│       ├── evaluate_models.py            # Benchmark all models
│       └── export_onnx.py               # Export models to ONNX for fast
inference
│
├── database/
│   ├── schema.sql                      # Full Supabase SQL migration
│   ├── seed.sql                        # Demo data seeding
│   ├── functions.sql                   # PostgreSQL functions
(aggregations, triggers)
│   └── policies.sql                    # Row Level Security policies
│
├── infra/
│   ├── docker-compose.yml              # Local dev: FastAPI + Redis +
Supabase
│   ├── docker-compose.prod.yml         # Production compose
│   └── vercel.json                     # Vercel frontend config
│
└── docs/
    ├── DEMO_SCRIPT.md                  # Exact demo sequence with timing
    ├── API_REFERENCE.md                # Full API documentation
    └── ARCHITECTURE.md                 # System architecture diagrams
```

1.2 Database Schema (Supabase PostgreSQL)

```sql
-- ============================================
-- MERCHANTS
-- ============================================
CREATE TABLE merchants (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    name TEXT NOT NULL,                          -- "Sunita Saree Shop"
    owner_name TEXT NOT NULL,                    -- "Sunita Devi"
    phone TEXT UNIQUE NOT NULL,
    business_type TEXT NOT NULL,                 -- "saree_shop", "kirana
    location JSONB,                              -- {city, state, pincode
    employee_count INTEGER DEFAULT 1,
    monthly_rent DECIMAL(12,2),
    paytm_merchant_id TEXT,                     -- Paytm ecosystem link
    payscore INTEGER DEFAULT 50,                -- 0-100 credit score
    payscore_grade TEXT DEFAULT 'C',            -- A/B/C/D/F
    preferred_language TEXT DEFAULT 'hi',        -- ISO 639-1
    morning_briefing_time TIME DEFAULT '09:00',
```

```
    onboarding_complete BOOLEAN DEFAULT FALSE,
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- TRANSACTIONS (unified income + expense ledger)
-- ==============================================
CREATE TABLE transactions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    type TEXT NOT NULL CHECK (type IN ('income', 'expense')),
    amount DECIMAL(12,2) NOT NULL CHECK (amount > 0),
    category TEXT NOT NULL,                        -- "upi_payment", "cash_
    subcategory TEXT,                              -- More specific classif
    description TEXT,                              -- Raw voice command or
    source TEXT NOT NULL DEFAULT 'voice',          -- "voice", "paytm_webhc
    payment_mode TEXT,                             -- "upi", "card", "walle
    customer_name TEXT,                            -- For income: who paid
    customer_phone TEXT,
    supplier_name TEXT,                            -- For expense: paid to
    hsn_code TEXT,                                 -- GST HSN/SAC code (aut
    gst_rate DECIMAL(4,2),                        -- Applicable GST rate
    gst_eligible BOOLEAN DEFAULT FALSE,           -- ITC eligible?
    is_recurring BOOLEAN DEFAULT FALSE,           -- Auto-detected recurrir
    recurring_frequency TEXT,                      -- "monthly", "weekly",
    voice_transcript TEXT,                         -- Original Hindi voice
    intent_confidence DECIMAL(4,3),               -- NLU confidence score
    metadata JSONB DEFAULT '{}',                  -- Flexible extra data
    recorded_at TIMESTAMPTZ DEFAULT now(),         -- When the transaction
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- UDHARI (Informal Credit/Receivables)
-- ==============================================
CREATE TABLE udhari (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    debtor_name TEXT NOT NULL,                     -- "Sharma ji"
    debtor_phone TEXT,
    debtor_paytm_id TEXT,                         -- If available
    amount DECIMAL(12,2) NOT NULL CHECK (amount > 0),
    amount_paid DECIMAL(12,2) DEFAULT 0,
```

```
    remaining DECIMAL(12,2) GENERATED ALWAYS AS (amount - amount_paid) ST
    status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'partial',
    risk_score DECIMAL(4,3),                      -- ML-predicted collectic
    optimal_channel TEXT,                          -- RL-learned best chanr
    optimal_timing TEXT,                           -- RL-learned best time
    optimal_tone TEXT,                             -- RL-learned best tone
    reminder_count INTEGER DEFAULT 0,
    last_reminder_at TIMESTAMPTZ,
    last_reminder_response TEXT,                   -- "paid", "ignored", "r
    payment_link TEXT,                             -- Paytm payment link UR
    payment_link_clicks INTEGER DEFAULT 0,
    escalation_level INTEGER DEFAULT 0,           -- 0-4 (friendly -> urger
    max_escalation INTEGER DEFAULT 3,             -- Merchant-set maximum
    source TEXT DEFAULT 'voice',                   -- "voice", "ocr_photo",
    notes TEXT,
    due_date DATE,
    created_at TIMESTAMPTZ DEFAULT now(),
    settled_at TIMESTAMPTZ,
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- CUSTOMERS (Auto-built from transactions)
-- ==============================================
CREATE TABLE customers (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    phone TEXT,
    paytm_id TEXT,
    total_visits INTEGER DEFAULT 1,
    total_spent DECIMAL(12,2) DEFAULT 0,
    average_order_value DECIMAL(12,2),
    first_visit TIMESTAMPTZ DEFAULT now(),
    last_visit TIMESTAMPTZ DEFAULT now(),
    visit_frequency_days DECIMAL(6,1),            -- Average days between v
    expected_next_visit DATE,                      -- Predicted from patter
    -- RFM Scoring
    rfm_recency_score INTEGER,                    -- 1-5
    rfm_frequency_score INTEGER,                  -- 1-5
    rfm_monetary_score INTEGER,                   -- 1-5
    rfm_segment TEXT,                             -- "champion", "loyal", "
    -- Churn Detection (TS2Vec)
    churn_risk TEXT DEFAULT 'low' CHECK (churn_risk IN ('low', 'medium',
```

```
    churn_probability DECIMAL(4,3),             -- 0.000 - 1.000
    churn_detected_at TIMESTAMPTZ,
    days_since_last_visit INTEGER,
    -- Winback
    winback_sent BOOLEAN DEFAULT FALSE,
    winback_message TEXT,
    winback_response TEXT,
    winback_offer TEXT,                          -- "10% discount", "free
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now(),
    updated_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- EMPLOYEES
-- ==============================================
CREATE TABLE employees (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    name TEXT NOT NULL,
    phone TEXT,
    role TEXT,                                   -- "helper", "cashier",
    salary DECIMAL(12,2),
    payment_frequency TEXT DEFAULT 'monthly',
    last_paid_date DATE,
    last_paid_amount DECIMAL(12,2),
    bank_account TEXT,                          -- UPI ID or account for
    attendance_this_month INTEGER DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- DAILY SUMMARY (Pre-aggregated for fast dashboard)
-- ==============================================
CREATE TABLE daily_summary (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    date DATE NOT NULL,
    total_income DECIMAL(12,2) DEFAULT 0,
    total_expense DECIMAL(12,2) DEFAULT 0,
    net_profit DECIMAL(12,2) DEFAULT 0,
    profit_margin DECIMAL(5,2),                 -- Percentage
    transaction_count INTEGER DEFAULT 0,
    unique_customers INTEGER DEFAULT 0,
```

```
    digital_income DECIMAL(12,2) DEFAULT 0,      -- UPI/card/wallet
    cash_income DECIMAL(12,2) DEFAULT 0,         -- Voice-logged cash
    udhari_created DECIMAL(12,2) DEFAULT 0,
    udhari_collected DECIMAL(12,2) DEFAULT 0,
    top_expense_category TEXT,
    top_customer TEXT,
    anomaly_flag BOOLEAN DEFAULT FALSE,          -- Revenue anomaly detect
    anomaly_description TEXT,
    UNIQUE(merchant_id, date)
);

-- ==============================================
-- EVENTS (Real-time activity feed)
-- ==============================================
CREATE TABLE events (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    event_type TEXT NOT NULL,                     -- See enum below
    title TEXT NOT NULL,                          -- "Rs 5,000 rent loggec
    title_hindi TEXT,                             -- "Rs 5,000 rent mein c
    description TEXT,
    severity TEXT DEFAULT 'info',                 -- "info", "success", "w
    related_entity_type TEXT,                     -- "transaction", "udhar
    related_entity_id UUID,
    metadata JSONB DEFAULT '{}',
    created_at TIMESTAMPTZ DEFAULT now()
);
-- event_types: income_added, expense_added, udhari_created, udhari_colle
--   udhari_partial, reminder_sent, reminder_response, payscore_change,
--   churn_detected, winback_sent, winback_success, gst_alert, gst_filed,
--   cash_crunch_alert, forecast_update, scheme_match, anomaly_detected

-- ==============================================
-- GST STATUS
-- ==============================================
CREATE TABLE gst_status (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    gstin TEXT,                                   -- GSTIN number
    return_type TEXT NOT NULL,                    -- "GSTR-1", "GSTR-3B"
    period TEXT NOT NULL,                         -- "March 2026"
    due_date DATE NOT NULL,
    status TEXT DEFAULT 'pending' CHECK (status IN ('pending', 'classifyi
    estimated_tax DECIMAL(12,2),
```

```
    itc_claimed DECIMAL(12,2),
    itc_matched DECIMAL(12,2),
    itc_mismatch DECIMAL(12,2),
    transactions_classified INTEGER DEFAULT 0,
    transactions_total INTEGER DEFAULT 0,
    classification_accuracy DECIMAL(4,3),
    filed_at TIMESTAMPTZ,
    penalty_amount DECIMAL(12,2) DEFAULT 0,
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- CASH FLOW FORECASTS
-- ==============================================
CREATE TABLE forecasts (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    forecast_date DATE NOT NULL,                -- Date this forecast is
    predicted_income DECIMAL(12,2),
    predicted_expense DECIMAL(12,2),
    predicted_net DECIMAL(12,2),
    confidence_upper DECIMAL(12,2),
    confidence_lower DECIMAL(12,2),
    is_festival BOOLEAN DEFAULT FALSE,
    festival_name TEXT,
    is_crisis BOOLEAN DEFAULT FALSE,            -- Net below threshold
    crisis_severity TEXT,                        -- "mild", "moderate", "
    model_version TEXT,                          -- "tft_v1", "chronos_v1
    generated_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- PAYSCORE HISTORY
-- ==============================================
CREATE TABLE payscore_history (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    score INTEGER NOT NULL,
    grade TEXT NOT NULL,
    feature_breakdown JSONB NOT NULL,           -- {consistency: 18, grow
    improvement_tips JSONB,                     -- [{tip: "File GST on ti
    credit_eligibility JSONB,                   -- {max_amount: 200000, r
    model_version TEXT,
    calculated_at TIMESTAMPTZ DEFAULT now()
```

```
);

-- ==============================================
-- COLLECTION ACTIONS (RL Agent Memory)
-- ==============================================
CREATE TABLE collection_actions (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    udhari_id UUID REFERENCES udhari(id) ON DELETE CASCADE,
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    action_type TEXT NOT NULL,                    -- "whatsapp", "sms", "v
    tone TEXT NOT NULL,                           -- "friendly_reminder" .
    message_text TEXT NOT NULL,                   -- The actual Hindi mess
    message_variant TEXT,                         -- A/B test variant ider
    sent_at TIMESTAMPTZ DEFAULT now(),
    delivered BOOLEAN DEFAULT FALSE,
    read BOOLEAN DEFAULT FALSE,
    read_at TIMESTAMPTZ,
    response TEXT,                                -- "paid", "partial_paic
    payment_received DECIMAL(12,2) DEFAULT 0,
    reward DECIMAL(6,3),                         -- RL reward signal
    state_features JSONB,                        -- State vector at time c
    created_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- GOVERNMENT SCHEMES
-- ==============================================
CREATE TABLE scheme_matches (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    scheme_name TEXT NOT NULL,
    scheme_code TEXT,                             -- "MUDRA_SHISHU", "PMEG
    description TEXT,
    eligible_amount DECIMAL(12,2),
    interest_rate DECIMAL(4,2),
    eligibility_score DECIMAL(4,3),              -- How well the merchant
    status TEXT DEFAULT 'matched',               -- "matched", "applied",
    application_url TEXT,
    matched_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- WHATSAPP MESSAGES (Full conversation log)
-- ==============================================
```

```
CREATE TABLE whatsapp_messages (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    merchant_id UUID REFERENCES merchants(id) ON DELETE CASCADE,
    direction TEXT NOT NULL CHECK (direction IN ('inbound', 'outbound')),
    recipient_type TEXT NOT NULL,                -- "merchant", "debtor",
    recipient_phone TEXT,
    recipient_name TEXT,
    message_type TEXT NOT NULL,                  -- "text", "voice", "ima
    content TEXT NOT NULL,
    template_name TEXT,                          -- For WhatsApp Business
    media_url TEXT,                              -- For voice notes, imag
    payment_link TEXT,                           -- Embedded Paytm link i
    status TEXT DEFAULT 'sent',                  -- "sent", "delivered",
    metadata JSONB DEFAULT '{}',
    sent_at TIMESTAMPTZ DEFAULT now()
);

-- ==============================================
-- INDEXES for performance
-- ==============================================
CREATE INDEX idx_transactions_merchant_date ON transactions(merchant_id,
CREATE INDEX idx_transactions_type ON transactions(merchant_id, type, rec
CREATE INDEX idx_transactions_category ON transactions(merchant_id, categ
CREATE INDEX idx_udhari_merchant_status ON udhari(merchant_id, status);
CREATE INDEX idx_udhari_overdue ON udhari(merchant_id, status, created_at
CREATE INDEX idx_customers_merchant_churn ON customers(merchant_id, churr
CREATE INDEX idx_customers_last_visit ON customers(merchant_id, last_visi
CREATE INDEX idx_events_merchant_time ON events(merchant_id, created_at D
CREATE INDEX idx_daily_summary_merchant ON daily_summary(merchant_id, dat
CREATE INDEX idx_forecasts_merchant ON forecasts(merchant_id, forecast_da
CREATE INDEX idx_collection_actions_udhari ON collection_actions(udhari_i
CREATE INDEX idx_whatsapp_merchant ON whatsapp_messages(merchant_id, sent

-- ==============================================
-- TRIGGERS: Auto-update daily_summary on new transaction
-- ==============================================
CREATE OR REPLACE FUNCTION update_daily_summary()
RETURNS TRIGGER AS $$
BEGIN
    INSERT INTO daily_summary (merchant_id, date, total_income, total_exp
    VALUES (
        NEW.merchant_id,
        DATE(NEW.recorded_at),
        CASE WHEN NEW.type = 'income' THEN NEW.amount ELSE 0 END,
```

```
        CASE WHEN NEW.type = 'expense' THEN NEW.amount ELSE 0 END,
        CASE WHEN NEW.type = 'income' THEN NEW.amount ELSE -NEW.amount EN
        1
    )
    ON CONFLICT (merchant_id, date) DO UPDATE SET
        total_income = daily_summary.total_income + CASE WHEN NEW.type =
        total_expense = daily_summary.total_expense + CASE WHEN NEW.type
        net_profit = daily_summary.net_profit + CASE WHEN NEW.type = 'inc
        transaction_count = daily_summary.transaction_count + 1,
        profit_margin = CASE
            WHEN (daily_summary.total_income + CASE WHEN NEW.type = 'incc
            THEN ((daily_summary.net_profit + CASE WHEN NEW.type = 'incom
                  (daily_summary.total_income + CASE WHEN NEW.type = 'inc
            ELSE 0
        END;
    RETURN NEW;
END;
$$ LANGUAGE plpgsql;

CREATE TRIGGER trigger_update_daily_summary
    AFTER INSERT ON transactions
    FOR EACH ROW
    EXECUTE FUNCTION update_daily_summary();
```

1.3 Tech Stack (Full Production)

Layer	Technology	Why
Frontend	Next.js 14 (App Router) + TypeScript	SSR, file-based routing, React Server Components
Styling	Tailwind CSS + shadcn/ui	Rapid premium UI, consistent design system
Animations	Framer Motion	Spring-based number morphing, layout animations
Charts	Recharts + D3.js	CashFlowChart needs custom festival markers (D3), rest via Recharts
Real-time	Socket.IO (client)	WebSocket with auto-reconnect + room subscriptions
Backend	FastAPI (Python 3.11+)	Async, type-safe, ML ecosystem compatibility
Real-time Server	python-socketio mounted on FastAPI	Bidirectional WebSocket events
Task Queue	Celery + Redis (or BullMQ)	Async jobs: collection reminders, forecasting, GST batch

Layer	Technology	Why
Database	Supabase (PostgreSQL 15)	Managed Postgres, auth, storage, free tier
Cache	Redis (Upstash serverless)	Real-time state, pub/sub, rate limiting, session
Vector DB	ChromaDB	RAG embeddings for government schemes + GST codes
Object Storage	Supabase Storage / S3	Voice recordings, OCR images, ML model artifacts
Auth	Supabase Auth (phone OTP)	Indian merchants → phone-first auth
Hosting (FE)	Vercel	Edge network, instant deploys, free tier
Hosting (BE)	Railway / Render	Docker container hosting, free/hobby tier
CI/CD	GitHub Actions	Automated lint, test, build, deploy

1.4 AI/ML Stack (Research-Grade)

Component	Model / Paper	Details
Speech-to-Text	IndicWhisper (AI4Bharat, 2024) — Whisper-v3 fine-tuned on IndicVoices 7K+ hrs	12% WER on Hindi, ~200ms latency. Fallback: Groq Whisper large-v3 API
Intent Classification	IndicBERTv2 (AI4Bharat, 2023) fine-tuned on 500+ Hindi financial commands	88% F1, 12 intent classes, ~50ms. Below 0.85 confidence → Groq LLM fallback
NER + Hindi Numerals	spaCy 3.x custom NER + rule-based Hindi numeral parser (40+ patterns)	Entities: AMOUNT, PERSON, CATEGORY, DATE. Handles "dedh lakh", "sawa do sau"
Code-Switching	HingBERT (AAAI 2021) — BERT pre-trained on Hindi-English code-mixed data	65.8% F1 on LinCE benchmark. Handles Hinglish seamlessly
Master Agent (LLM)	Groq LPU: Mixtral 8x7B at 500 tok/sec + Llama 3.1 70B	Constitutional AI guardrails (RBI compliance). LangGraph multi-agent orchestration

Component	Model / Paper	Details
Cash Flow Forecasting	Temporal Fusion Transformer (Lim et al., Google, 2021) + Amazon Chronos (2024)	TFT: multi-horizon attention with static covariates + known future inputs (festivals). Chronos: zero-shot foundation model. Ensemble: weighted by horizon length
Credit Scoring (PayScore)	TabNet (Arik & Pfister, 2021) + HAN Graph (Yang et al., 2016)	TabNet: 47 features with interpretable sequential attention — merchant sees WHICH features helped/hurt. HAN: models account relationships from Account Aggregator data as a graph. Augmented with transaction embeddings
Churn Detection	TS2Vec (Yue et al., AAAI 2022) contrastive pre-training	Universal time-series representation learning. Pre-train on 500K unlabeled transaction sequences, fine-tune classifier on labeled churn data. 0.89 AUROC vs 0.76 with standard approaches
Collection Optimization	Thompson Sampling (multi-armed bandit) → DQN (Mnih et al., Nature 2015)	Per-debtor strategy learning. State: (days_overdue, amount, past_responses, debtor_profile, time, day). Action: (channel, tone, timing). Reward: payment_speed * amount - annoyance_penalty
GST Classification	IndicBERTv2 fine-tuned on 18K+ HSN/SAC codes	94%+ accuracy on common merchant categories. Maps transactions to HSN codes automatically
Scheme Matching (RAG)	CRAG (Corrective RAG, Yan et al., 2024) + BGE-M3 (BAAI, 2024)	BGE-M3: dense + sparse + multi-granularity retrieval. Cross-lingual: Hindi query matches English scheme docs. CRAG: self-correcting retrieval with knowledge refinement
Hindi TTS	Sarvam Bulbul (Sarvam AI, 2024) — 2B parameter Indic TTS	Natural Hindi voice output. "Muneem personality" — respectful, concise, business-savvy
Paper Udhari OCR	PaddleOCR v4 (PaddlePaddle, 2024)	Hindi handwriting recognition. 85%+ accuracy on semi-legible paper ledgers
Noise Reduction	noisereduce (spectral gating) + WebRTC VAD	Shop noise filtering (traffic, TV, conversations) before STT

PHASE 2: AI/ML Engine — The Brain

2.1 Voice-First Financial NLU Pipeline

Architecture (< 670ms total latency):

```
Audio Input (Soundbox mic / WhatsApp voice / Browser mic)
    │
    ├─ [~50ms] Noise Reduction (noisereduce + WebRTC VAD)
    │          Filters: shop noise, traffic, TV, customer chatter
    │
    ├─ [~200ms] IndicWhisper STT (AI4Bharat)
    │           Fine-tuned Whisper-v3 on 7K+ hrs IndicVoices
    │           12% WER Hindi | Handles noisy shop environments
    │           Output: "Muneem, Sharma ji ka 8000 udhari hai"
    │
    ├─ [~50ms] IndicBERTv2 Intent Classifier
    │          Fine-tuned on 500+ Hindi financial command variations
    │          12 intent classes | 88% F1
    │          Output: {intent: "UDHARI_CREATE", confidence: 0.96}
    │          If confidence < 0.85 → Groq LLM fallback (Mixtral 8x7B)
    │
    ├─ [~30ms] spaCy NER + Hindi Numeral Parser
    │          Custom entities: AMOUNT, PERSON, CATEGORY, DATE
    │          Hindi numeral handling:
    │            "aath hazaar" → 8,000
    │            "dedh lakh" → 1,50,000
    │            "paanch sau" → 500
    │            "sawa do sau" → 225
    │            "dhai hazaar" → 2,500
    │          Output: {amount: 8000, person: "Sharma ji", type:
"UDHARI_CREATE"}
    │
    ├─ [~15ms] PostgreSQL Write (Supabase)
    │          INSERT into udhari/transactions table
    │
    ├─ [~5ms]  Redis PUBLISH
    │          Push event to real-time channel
    │
    ├─ [~10ms] Socket.IO Event Router
    │          Emit to merchant's dashboard room
    │
    └─ [~300ms] Framer Motion Dashboard Animation
               Number morphs, color transitions, activity feed update
```

Intent Classification — 12 Classes:

Intent	Example Commands (Hindi)	Action

Intent	Example Commands (Hindi)	Action
CASH_RECEIVED	"Rs 800 cash mila", "paanch sau rupaye aaye"	Log income (cash)
EXPENSE_LOG	"Rs 5000 rent diya", "bijli ka bill 2200"	Log expense
UDHARI_CREATE	"Sharma ji ka 8000 udhari", "Tripathi ji ko 12000 diya udhar"	Create udhari entry
UDHARI_SETTLE	"Sharma ji ne 5000 wapas kiya", "Mehra ji ka udhari khatam"	Settle/partial settle
QUERY_SUMMARY	"aaj kaisa raha", "aaj ka hisaab bata"	Today's P&L summary
QUERY_PROFIT	"profit kitna hua", "munafa bata"	Profit calculation
QUERY_EXPENSE	"rent kitna diya", "sabse zyada kharcha kahan"	Expense breakdown
QUERY_CUSTOMER	"Mehra ji ka record", "kitne customer aaye aaj"	Customer data
COMMAND_REMIND	"Sharma ji ko remind karo", "sab ko yaad dilao"	Trigger collection
COMMAND_GST	"GST file karo", "tax kitna bacha"	GST actions
PAYMENT_TAG	"abhi wali payment Sharma ji ki thi"	Tag last payment to customer
GENERAL	Anything else	Route to Groq LLM for conversational response

Hindi Numeral Parser — 40+ Patterns:

```
HINDI_NUMERAL_MAP = {
    # Direct numbers
    "ek": 1, "do": 2, "teen": 3, "chaar": 4, "paanch": 5,
    "chhah": 6, "saat": 7, "aath": 8, "nau": 9, "das": 10,
    "gyarah": 11, "barah": 12, "terah": 13, "chaudah": 14, "pandrah": 15,
    "solah": 16, "satrah": 17, "athaarah": 18, "unees": 19, "bees": 20,
    "pachees": 25, "tees": 30, "chalees": 40, "pachaas": 50,
    "saath": 60, "sattar": 70, "assi": 80, "nabbe": 90, "sau": 100,

    # Compound patterns
    "hazaar": 1000, "lakh": 100000, "crore": 10000000,
    "dedh": 1.5,           # dedh lakh = 1,50,000 | dedh sau = 150
    "dhai": 2.5,           # dhai hazaar = 2,500 | dhai lakh = 2,50,000
    "sawa": 1.25,          # sawa sau = 125 | sawa do sau = 225
    "paune": 0.75,         # paune do sau = 175
    "saadhe": lambda n: n + 0.5,  # saadhe teen sau = 350
```

```
    # Common spoken shortcuts
    "paanch sau": 500,
    "aath sau": 800,
    "do hazaar": 2000,
    "paanch hazaar": 5000,
    "das hazaar": 10000,
    "pandrah hazaar": 15000,
    "bees hazaar": 20000,
    "pachaas hazaar": 50000,
    "ek lakh": 100000,
    "dedh lakh": 150000,
    "do lakh": 200000,
    "paanch lakh": 500000,
}
```

2.2 Master Agent — LangGraph Multi-Agent Orchestration

```
# services/ai-engine/services/agents/master_agent.py

"""
MunimAI Master Agent — LangGraph Orchestration

Architecture: Master Agent (Groq LPU) orchestrates 7 specialist agents.
Each specialist handles one domain. Master routes, coordinates, and
synthesizes responses with the "Muneem personality."

Constitutional AI Guardrails:
- RBI Fair Practices Code compliance for collection messages
- No threatening language in any debtor communication
- Financial advice disclosures ("ye suggestion hai, financial advisor se
- Data privacy compliance (no debtor data shared externally)
"""

from langgraph.graph import StateGraph, END
from langgraph.prebuilt import ToolNode

class MunimState(TypedDict):
    merchant_id: str
    input_text: str
    input_source: str          # "soundbox", "whatsapp", "dashboard"
    intent: str
```

```
    entities: dict
    confidence: float
    context: dict              # Merchant profile, recent history
    specialist_results: list   # Results from specialist agents
    response_hindi: str        # Final Hindi response
    actions_taken: list        # DB writes, messages sent, etc.
    dashboard_delta: dict      # Changes to push via WebSocket

# The LangGraph state machine:
# 1. NLU → classify intent + extract entities
# 2. Route → select specialist agent(s)
# 3. Execute → specialist performs domain action
# 4. Synthesize → Master generates Hindi response
# 5. Emit → Push WebSocket events + TTS response

SPECIALIST_ROUTING = {
    "CASH_RECEIVED":    ["action_router"],           # Direct DB write
    "EXPENSE_LOG":      ["action_router"],            # Direct DB write
    "UDHARI_CREATE":    ["action_router", "collection_agent"],  # Create
    "UDHARI_SETTLE":    ["action_router", "payscore_agent"],    # Settle
    "QUERY_SUMMARY":    ["cashflow_agent"],            # Aggregate + fore
    "QUERY_PROFIT":     ["action_router"],             # Simple aggregati
    "COMMAND_REMIND":   ["collection_agent"],           # Trigger collect
    "COMMAND_GST":      ["gst_agent"],                 # GST actions
    "GENERAL":          ["master_llm"],                # Free-form conver
}
```

2.3 Temporal Fusion Transformer — Cash Flow Forecasting

Paper: "Temporal Fusion Transformers for Interpretable Multi-Horizon Time Series Forecasting" (Lim et al., Google Research, 2021)

Why TFT over Prophet/LSTM:

- Multi-horizon: simultaneously forecasts 30/60/90 days (not separate models)
- Interpretable: attention weights show WHICH past events drive each forecast day
- Static covariates: encodes business_type, location, employee_count as static features
- Known future inputs: festival_calendar, GST_deadlines, rent_due_dates fed as known future inputs
- Variable selection: automatically learns which of the 47 input features matter most

Implementation:

```python
# services/ml-training/train_tft_forecaster.py

from pytorch_forecasting import TemporalFusionTransformer, TimeSeriesData
from pytorch_forecasting.data import GroupNormalizer

class MunimTFTForecaster:
    """
    Inputs per merchant:
    - Time-varying observed: daily_income, daily_expense, udhari_created,
                             customer_count, digital_ratio, avg_transacti
    - Time-varying known future: is_festival (30+ festivals), is_weekend,
                                 gst_due_in_days, rent_due_in_days, fest
    - Static: business_type, city_tier, employee_count, months_on_platfor

    Output: 90-day forecast with prediction intervals
    - daily_predicted_income (with confidence bands)
    - daily_predicted_expense
    - daily_net_cashflow
    - crisis_alerts (dates where predicted_net < merchant's threshold)
    - festival_impact_scores (how much each upcoming festival affects for
    """

    def build_dataset(self, merchant_data: pd.DataFrame) -> TimeSeriesDat
        return TimeSeriesDataSet(
            merchant_data,
            time_idx="day_index",
            target="net_cashflow",
            group_ids=["merchant_id"],
            max_encoder_length=90,           # Look back 90 days
            max_prediction_length=90,        # Predict 90 days ahead
            static_categoricals=["business_type", "city_tier"],
            static_reals=["employee_count", "months_on_platform"],
            time_varying_known_categoricals=["is_festival", "is_weekend",
            time_varying_known_reals=["gst_due_in_days", "rent_due_in_day
            time_varying_unknown_categoricals=[],
            time_varying_unknown_reals=[
                "daily_income", "daily_expense", "udhari_created",
                "udhari_collected", "customer_count", "digital_ratio"
            ],
            target_normalizer=GroupNormalizer(groups=["merchant_id"]),
        )

    def train(self, dataset: TimeSeriesDataSet):
        trainer = pl.Trainer(max_epochs=50, gradient_clip_val=0.1)
```

```
    tft = TemporalFusionTransformer.from_dataset(
        dataset,
        hidden_size=64,
        attention_head_size=4,
        dropout=0.1,
        hidden_continuous_size=16,
        loss=QuantileLoss(),           # Produces prediction interval
        learning_rate=0.001,
        reduce_on_plateau_patience=4,
    )
    trainer.fit(tft, train_dataloaders=train_loader, val_dataloaders=
    return tft
```

Chronos Ensemble:

```
# Amazon Chronos (2024) — Zero-shot foundation model for time series
from chronos import ChronosPipeline

class ChronosForecaster:
    """
    Chronos is pre-trained on billions of time series.
    Zero-shot: works without fine-tuning on merchant-specific data.
    Best for: new merchants with < 3 months of data (cold start problem).

    Ensemble strategy:
    - Merchant has >= 6 months data → TFT weight: 0.7, Chronos weight: 0.
    - Merchant has 3-6 months → TFT: 0.5, Chronos: 0.5
    - Merchant has < 3 months → TFT: 0.0, Chronos: 1.0 (cold start)
    """
    def __init__(self):
        self.pipeline = ChronosPipeline.from_pretrained("amazon/chronos-t

    def predict(self, history: torch.Tensor, horizon: int = 90):
        forecast = self.pipeline.predict(history, horizon)
        return {
            "median": forecast.median(dim=1),
            "lower": forecast.quantile(0.1, dim=1),
            "upper": forecast.quantile(0.9, dim=1),
        }
```

2.4 TabNet + HAN — PayScore Credit Engine

Paper (TabNet): "TabNet: Attentive Interpretable Tabular Learning" (Arik & Pfister, Google Cloud AI, 2021)
Paper (HAN): "Hierarchical Attention Networks" (Yang et al., 2016)

Why TabNet over XGBoost/LightGBM:

- **Interpretable sequential attention:** Each prediction step selects a SUBSET of features via soft attention → merchant sees exactly WHY their score changed ("Your udhari_collection_rate dropped → -3 points")
- **Built-in feature selection:** No manual feature engineering needed
- **Handles missing data natively:** Account Aggregator data may be partially available
- **End-to-end gradient descent:** No separate feature engineering pipeline

Why HAN Graph augmentation:

- Account Aggregator data creates a GRAPH of financial relationships (merchant → banks → suppliers → customers)
- HAN captures hierarchical attention: word-level → sentence-level analogous to transaction-level → account-level
- Models relationships between accounts that flat tabular models miss

47-Feature PayScore Implementation:

```
# services/ai-engine/services/ml/tabnet_scorer.py

from pytorch_tabnet.tab_model import TabNetClassifier
import torch

class PayScoreEngine:
    FEATURE_GROUPS = {
        "consistency": [                          # Weight: 25%
            "revenue_coefficient_of_variation",  # Lower CV = more stable
            "days_with_zero_revenue_pct",        # % of days with no trar
            "weekly_pattern_strength",            # Autocorrelation at la
            "longest_no_revenue_streak",          # Max consecutive zero
            "payment_mode_diversity_entropy",     # Shannon entropy of pa
            "daily_revenue_std_normalized",       # Std/mean of daily rev
            "weekend_weekday_consistency",        # Revenue ratio weekday
        ],
        "growth": [                               # Weight: 20%
            "revenue_3m_slope",                   # Linear regression slop
            "revenue_6m_slope",                   # Linear regression slop
            "customer_base_growth_rate",          # New unique customers
            "avg_transaction_value_trend",        # AOV slope over time
```

```
        "digital_adoption_rate_trend",        # % digital payments tr
        "month_over_month_revenue_growth",    # MoM growth rate
        "customer_retention_rate",            # % returning customers
    ],
    "risk": [                                # Weight: 20%
        "customer_herfindahl_index",          # Revenue concentratior
        "seasonal_vulnerability_index",       # Max-min revenue ratic
        "single_day_revenue_dependency",      # % from best day of we
        "udhari_to_revenue_ratio",           # Outstanding receivable
        "udhari_collection_rate_30d",        # % collected within 30
        "bad_debt_write_off_ratio",          # Historical write-offs
        "expense_to_revenue_ratio_trend",    # Is margin shrinking?
        "cash_burn_rate_if_negative",        # Days of cash runway at
    ],
    "discipline": [                          # Weight: 15%
        "gst_filing_timeliness_score",       # On-time filing %
        "gst_itc_mismatch_frequency",        # How often ITC mismatch
        "supplier_payment_timeliness",        # Auto-detected from ex
        "expense_logging_consistency",        # How regularly they us
        "digital_record_completeness",        # % of transactions dig
        "munimAI_daily_active_usage",         # How engaged with the
    ],
    "depth": [                               # Weight: 10%
        "months_on_paytm_platform",          # Tenure
        "total_lifetime_gmv",                # All-time transaction v
        "paytm_product_breadth",             # QR + Soundbox + Lendir
        "transaction_velocity_trend",         # Transactions per day
        "business_hours_consistency",         # Regular operating hou
    ],
    "account_aggregator": [                  # Weight: 10% (when avai
        "bank_balance_stability_30d",        # Std of daily closing b
        "bank_inflow_outflow_ratio",         # Monthly inflows / outf
        "emi_to_income_ratio",               # Fixed obligations / re
        "savings_rate",                       # % of income retained
        "cross_verified_turnover_accuracy",   # Paytm GMV vs GSTN tur
        "other_loan_obligations_count",       # Outstanding loans fro
    ],
}

def train(self, features_df: pd.DataFrame, labels: pd.Series):
    """
    Labels: 5-class credit grade (A/B/C/D/F) derived from:
    - Actual default events (from Paytm lending data)
    - Cash flow sustainability index
```

```
        - Business growth trajectory
        """
        self.model = TabNetClassifier(
            n_d=32, n_a=32,                        # Decision/attention embe
            n_steps=5,                              # Number of sequential a
            gamma=1.5,                              # Coefficient for featur
            n_independent=2,
            n_shared=2,
            momentum=0.02,
            mask_type='entmax',                     # Sparse attention (bett
        )
        self.model.fit(
            X_train=features_df.values,
            y_train=labels.values,
            eval_set=[(X_val.values, y_val.values)],
            max_epochs=200,
            patience=20,
            batch_size=256,
        )

    def explain_score(self, merchant_features: np.ndarray) -> dict:
        """
        TabNet's built-in feature importance per prediction.
        Returns which features helped and which hurt.
        Translated to Hindi tips:
          "Aapka udhari_collection_rate kam hai → -4 points.
           Agar Rs 50K udhari collect karo → +4 points possible"
        """
        importance = self.model.explain(merchant_features)
        feature_mask = importance[0]  # Per-step attention masks
        aggregate_importance = feature_mask.mean(axis=0)
        return self._generate_hindi_explanation(aggregate_importance)
```

2.5 TS2Vec — Customer Churn Detection

Paper: "TS2Vec: Towards Universal Representation of Time Series" (Yue et al., AAAI 2022)

Why TS2Vec over simple RFM thresholds:

- **Contrastive learning:** Learns universal representations of customer transaction sequences WITHOUT labels
- **Pre-train on 500K unlabeled sequences:** Leverage all Paytm transaction data, not just labeled churn events

- **Temporal contrast:** Learns both timestamp-level and instance-level representations
- **Achieves 0.89 AUROC** vs 0.76 with standard XGBoost on RFM features

```
# services/ai-engine/services/ml/ts2vec_embedder.py

from ts2vec import TS2Vec
import torch

class ChurnDetector:
    """
    Two-stage churn detection:

    Stage 1: TS2Vec Contrastive Pre-training (unsupervised)
    - Input: 500K customer transaction sequences (amount, frequency, time
    - Output: 320-dim embedding per customer capturing behavioral patterr
    - Training: Hierarchical contrastive loss over augmented time series

    Stage 2: Classification Head (supervised)
    - Input: TS2Vec embeddings + RFM features + static features
    - Output: churn probability (0-1) + days-to-churn estimate
    - Model: 2-layer MLP on top of frozen TS2Vec embeddings
    """

    def __init__(self):
        self.encoder = TS2Vec(
            input_dims=5,           # [amount, days_since_last, visit_cou
            output_dims=320,        # Embedding dimension
            hidden_dims=128,
            depth=10,               # Number of dilated conv layers
        )

    def pretrain(self, all_sequences: np.ndarray):
        """Pre-train on ALL customer sequences (no labels needed)"""
        loss_log = self.encoder.fit(
            all_sequences,
            n_epochs=200,
            batch_size=512,
            verbose=True,
        )
        return loss_log

    def encode_customer(self, customer_sequence: np.ndarray) -> np.ndarra
        """Get 320-dim embedding for a single customer's transaction hist
        return self.encoder.encode(customer_sequence, encoding_window='fu
```

```
    def detect_churn(self, embedding: np.ndarray, rfm_features: np.ndarra
        """
        Combined TS2Vec embedding + RFM → churn probability

        Returns:
        {
            "churn_probability": 0.73,
            "churn_risk": "high",
            "estimated_days_to_churn": 12,
            "top_signals": ["visit_frequency_dropping", "aov_declining",
            "recommended_action": "Send 15% discount offer via WhatsApp",
            "winback_urgency": "high"
        }
        """
        combined = np.concatenate([embedding, rfm_features])
        prob = self.classifier.predict_proba(combined)[1]
        return self._format_churn_result(prob, rfm_features)
```

2.6 Collection Engine — RL-Based Adaptive Udhari Recovery

Multi-Armed Bandit → Deep Q-Network progression:

```
# services/ai-engine/services/agents/collection_agent.py

class CollectionAgent:
    """
    Phase 1 (< 50 interactions per debtor): Thompson Sampling
    - Models each (channel, tone, timing) combination as a Beta distribut
    - Samples from posterior to balance exploration/exploitation
    - Lightweight, works with minimal data

    Phase 2 (>= 50 interactions): Deep Q-Network (DQN)
    - State: [days_overdue, amount, num_reminders, past_response_encoding
             debtor_digital_activity, time_of_day, day_of_week, festival
    - Action: (channel x tone x timing) = 3 x 5 x 4 = 60 discrete actions
    - Reward: payment_received * speed_bonus - annoyance_penalty
    - Experience replay buffer per debtor type

    Cultural Intelligence:
    - Always uses "ji" suffix
    - Never threatens ("ye last reminder hai" not "pay or else")
    - Respects business hours (no 6 AM messages)
```

```
- Adjusts tone to relationship (regular customer vs one-time buyer)
- RBI Fair Practices Code compliant (Constitutional AI guardrail)
"""

TONE_LEVELS = {
    0: "friendly_reminder",    # "Sharma ji, yaad dila raha hoon ki F
    1: "polite_follow_up",     # "Aapka account pending hai, convenie
    2: "firm_request",         # "Rs 8,000 pending hai 30 din se. Jal
    3: "urgent_notice",        # "Ye important reminder hai. Rs 8,00(
    4: "escalation_notice",    # "Sunita ji ko ye matter forward karr
}

TIMING_SLOTS = {
    "morning_9am": "09:00",    # Before shop rush — best for shopkeep
    "afternoon_2pm": "14:00",  # Post-lunch lull
    "evening_7pm": "19:00",    # After work
    "weekend_10am": "10:00",   # Saturday morning — leisure time
}

CHANNELS = ["whatsapp_text", "whatsapp_voice", "sms"]

def generate_collection_message(self, debtor: Debtor, tone: str) -> s
    """
    Uses Groq LLM to generate culturally appropriate Hindi messages.
    Each message is unique (no spam-like repetition).
    A/B variants tracked for effectiveness.
    """
    prompt = f"""You are MunimAI's collection assistant. Generate a F
    to collect Rs {debtor.remaining} from {debtor.name}.

    Tone: {tone}
    Days overdue: {debtor.days_overdue}
    Previous reminders: {debtor.reminder_count}
    Merchant name: {debtor.merchant_name}
    Paytm payment link: {debtor.payment_link}

    Rules:
    - Always use "ji" suffix with debtor name
    - Include the Paytm payment link naturally in the message
    - Never use threatening language
    - Be culturally sensitive — this is a social relationship, not ju
    - If first reminder, be very casual and friendly
    - Include the exact amount
    - Keep under 200 characters for WhatsApp readability
```

```
        - RBI Fair Practices compliant

        Generate ONLY the message text in Hindi. No explanation."""

        return groq_client.chat(prompt)
```

2.7 CRAG + BGE-M3 — Government Scheme Navigator

Paper (CRAG): "Corrective Retrieval Augmented Generation" (Yan et al., 2024)
Paper (BGE-M3): "BGE M3-Embedding: Multi-Lingual, Multi-Functionality, Multi-Granularity" (BAAI, 2024)

```
# services/ai-engine/services/agents/scheme_agent.py

class SchemeNavigator:
    """
    Cross-lingual RAG: Hindi merchant profile → matches English governmer

    BGE-M3 advantages:
    - Multi-lingual: embeds Hindi and English in same vector space
    - Multi-functionality: dense + sparse + colbert retrieval in one mode
    - Multi-granularity: sentence and passage level embeddings

    CRAG self-correction:
    - Retrieval → Evaluator judges relevance → If low confidence, trigger
    - Knowledge refinement: extracts only relevant sentences from retriev
    - Corrective: if initial retrieval is wrong, it CORRECTS before gener

    Pipeline:
    1. Build merchant eligibility profile from Paytm data
       {sector: "textile_retail", turnover: 4200000, employees: 3, years:
    2. BGE-M3 encodes profile as query vector
    3. ChromaDB retrieves top-5 matching schemes
    4. CRAG evaluator scores relevance of each
    5. LLM generates Hindi explanation of matched schemes
    6. Pre-fills application forms where possible
    """

    def __init__(self):
        self.embedder = BGEM3FlagModel('BAAI/bge-m3')
        self.vector_db = chromadb.Client()
        self.collection = self.vector_db.get_or_create_collection("govt_s
```

```python
    def index_schemes(self, schemes: List[dict]):
        """Index 50+ MSME government schemes"""
        for scheme in schemes:
            embedding = self.embedder.encode(
                scheme["description"],
                return_dense=True,
                return_sparse=True,
            )
            self.collection.add(
                documents=[scheme["description"]],
                embeddings=[embedding["dense_vecs"]],
                metadatas=[{
                    "name": scheme["name"],
                    "max_amount": scheme["max_amount"],
                    "interest_rate": scheme["interest_rate"],
                    "eligibility": json.dumps(scheme["eligibility"]),
                }],
                ids=[scheme["code"]],
            )

    def match_schemes(self, merchant_profile: dict) -> List[SchemeMatch]:
        """Find matching schemes for merchant"""
        query = self._build_query(merchant_profile)
        results = self.collection.query(query_embeddings=[self.embedder.e
        # CRAG evaluation step
        evaluated = self._crag_evaluate(results, merchant_profile)
        return [self._format_match(r, merchant_profile) for r in evaluate
```

PHASE 3: Frontend — Premium Dashboard

3.1 Design System

```typescript
// apps/web/src/lib/constants.ts

export const COLORS = {
    primary: '#00BAF2',         // Paytm blue
    primaryDark: '#002E6E',     // Navy
    success: '#22C55E',         // Green — income, positive
    warning: '#F59E0B',         // Amber — thin margin, medium risk
    danger: '#EF4444',          // Red — expense, negative, high risk
    purple: '#7C3AED',          // AI processing indicators
```

```
    background: '#F8FAFC',     // Very light gray
    cardBg: '#FFFFFF',
    textPrimary: '#1E293B',
    textSecondary: '#64748B',
    border: '#E2E8F0',
};

export const THRESHOLDS = {
    profitMargin: { good: 30, warning: 10 },  // green > 30%, yellow 10-3
    payScore: { excellent: 80, good: 60, fair: 40 },
    churnRisk: { high: 0.7, medium: 0.4 },
    udhariOverdue: 30,  // days
};

// Indian number formatting
export function formatINR(amount: number): string {
    if (amount >= 10000000) return `Rs ${(amount / 10000000).toFixed(1)}
    if (amount >= 100000) return `Rs ${(amount / 100000).toFixed(1)} L`;
    return `Rs ${amount.toLocaleString('en-IN')}`;
}
```

3.2 Hero Component — LivePnLCard

```
┌──────────────────────────────────────────────────────────────────────┐
│  [LIVE •] Aaj Ka Hisaab              31 March 2026                   │
│  ──────────────────────────────────────────────────────────────────  │
│                                                                      │
│   ↑ Kamaai (Income)     ↓ Kharcha (Expense)     📊 Munafa           │
│   Rs 34,500             Rs 12,400                Rs 22,100          │
│   +12% kal se           Rent ₹5K + Stock ₹7.4K  Margin: 64.1%      │
│                                                                      │
│   ▓▓▓
```

```
- Expense changes → red flash + scale 1.1x
- Profit going negative → entire card border pulses red
- Margin bar color transitions smoothly (green → yellow → red)
- Sound effects: ka-ching on income, whoosh on expense
```

3.3 Key Pages

Page	Route	Components	Real-time?
Dashboard	`/`	LivePnLCard, UdhariTracker, PayScoreGauge, CustomerPulse, CashFlowMini, GSTStatusBadge, ActivityFeed, AlertBanner, VoiceInput	Yes — WebSocket
Udhari	`/udhari`	Full debtor list, filters (overdue/pending/settled), bulk remind, individual debtor detail, collection history, A/B test results	Yes
Forecast	`/forecast`	CashFlowChart (30/60/90 day), festival markers, crisis zones (red shading), recommendations, anomaly highlights	Refreshes on new data
Customers	`/customers`	Customer list with RFM segments, churn risk badges, CustomerRFMScatter, individual profiles, winback campaign history	Yes
GST	`/gst`	Filing status timeline, transaction classification progress, ITC reconciliation, penalty tracker, "File GST" flow	On update
Schemes	`/schemes`	Matched scheme cards, eligibility score, application status, pre-filled forms	Static
Employees	`/employees`	Employee list, salary tracker, payment history, "Pay Salary" flow	On update
WhatsApp	`/whatsapp`	Full conversation view (merchant, debtors, customers), morning briefing, message templates	Yes
Soundbox	`/soundbox`	Soundbox simulator, LED indicator, voice command history, recent confirmations	Yes
Onboarding	`/onboarding`	Language selection, business profile, Paytm merchant linking, paper udhari photo import (OCR)	One-time

---

PHASE 4: Real-Time Infrastructure

4.1 WebSocket Events (Socket.IO)

Server → Client Events:

Event	Payload	Dashboard Animation
`dashboard_update`	`{field, old value, new_value, delta, animation_type}`	Number morph + color flash
`new transaction`	`{transaction object}`	Activity feed scroll + P&L update
`udhari_update`	`{udhari object, change_type}`	UdhariTracker item animates (create/settle/partial)
`payscore_change`	`{old score, new_score, delta, reason}`	Gauge animates, tooltip shows reason
`alert`	`{type, severity, message_hindi, actions}`	AlertBanner slides in from top
`collection_result`	`{debtor, amount_paid, status}`	Udhari item turns green, income updates
`churn_detected`	`{customer, probability, segment}`	CustomerPulse card highlights red
`voice_response`	`{transcript, intent, entities, response_hindi, audio_url}`	Transcript display + TTS playback
`forecast_update`	`{crisis_dates, recommendations}`	CashFlowMini sparkline updates
`gst_update`	`{status, days remaining, estimated_tax}`	GSTStatusBadge updates
`new_event`	`{event object}`	ActivityFeed prepends new item with slide animation

4.2 API Endpoints (Complete)

```
# ========================================
# VOICE PROCESSING (Critical Path)
# ========================================
POST   /api/voice/process              # Audio blob → STT → NLU → Action →
Response
POST   /api/voice/text                 # Text input → same NLU pipeline
(skip STT)

# ========================================
# DASHBOARD
# ========================================
GET    /api/dashboard/{merchant_id}              # Full dashboard state
```

```
GET    /api/dashboard/{merchant_id}/today         # Today's P&L only
GET    /api/dashboard/{merchant_id}/monthly        # Monthly aggregation
GET    /api/dashboard/{merchant_id}/comparison     # This month vs last
month

# ========================================
# TRANSACTIONS
# ========================================
POST   /api/transactions                           # Create transaction
GET    /api/transactions/{merchant_id}             # List with filters
(type, category, date range)
GET    /api/transactions/{merchant_id}/categories  # Category-wise
breakdown
DELETE /api/transactions/{id}                       # Delete (voice: "wo
entry hata do")

# ========================================
# UDHARI
# ========================================
GET    /api/udhari/{merchant_id}                   # List all (with
filters)
POST   /api/udhari                                 # Create udhari entry
PATCH  /api/udhari/{id}/settle                     # Full or partial
settlement
POST   /api/udhari/{id}/remind                     # Trigger single debtor
reminder
POST   /api/udhari/remind-all                      # Remind all overdue
GET    /api/udhari/{merchant_id}/stats             # Collection stats,
recovery rate
POST   /api/udhari/import-photo                    # OCR paper udhari photo

# ========================================
# PAYSCORE
# ========================================
GET    /api/payscore/{merchant_id}                 # Current score +
breakdown + tips
GET    /api/payscore/{merchant_id}/history         # Score over time
POST   /api/payscore/{merchant_id}/recalculate     # Force recalculation

# ========================================
# CASH FLOW FORECAST
```

```
# ========================================
GET    /api/forecast/{merchant_id}                # 30/60/90 day forecast
with festival markers
GET    /api/forecast/{merchant_id}/crisis          # Upcoming crisis dates
+ recommendations
POST   /api/forecast/{merchant_id}/retrain         # Retrain on latest
data

# ========================================
# GST
# ========================================
GET    /api/gst/{merchant_id}                     # Current GST filing
status
POST   /api/gst/{merchant_id}/classify             # Auto-classify
transactions to HSN codes
POST   /api/gst/{merchant_id}/prepare              # Prepare GSTR-1 /
GSTR-3B
POST   /api/gst/{merchant_id}/file                 # Submit filing (mock
for demo)
GET    /api/gst/{merchant_id}/itc-reconciliation    # ITC mismatch report

# ========================================
# CUSTOMERS
# ========================================
GET    /api/customers/{merchant_id}               # List with RFM + churn
risk
GET    /api/customers/{merchant_id}/at-risk        # High churn risk
customers
POST   /api/customers/{merchant_id}/winback/{customer_id}  # Send winback
offer
GET    /api/customers/{merchant_id}/segments        # RFM segment
distribution

# ========================================
# SCHEMES
# ========================================
GET    /api/schemes/{merchant_id}                 # Matched government
schemes
GET    /api/schemes/{merchant_id}/{scheme_code}    # Detailed scheme info
+ eligibility
POST   /api/schemes/{merchant_id}/{scheme_code}/apply  # Start application
```

```
(pre-fill)

# ========================================
# EMPLOYEES
# ========================================
GET    /api/employees/{merchant_id}                # List employees
POST   /api/employees                              # Add employee
PATCH  /api/employees/{id}                         # Update salary/details
POST   /api/employees/{id}/pay                     # Process salary payment

# ========================================
# WHATSAPP
# ========================================
POST   /api/whatsapp/send                          # Send message
GET    /api/whatsapp/{merchant_id}/messages         # Conversation history
POST   /api/whatsapp/briefing/{merchant_id}        # Generate + send
morning briefing
POST   /api/whatsapp/webhook                       # Meta Cloud API
incoming webhook

# ========================================
# BRIEFING
# ========================================
GET    /api/briefing/{merchant_id}/generate         # Generate morning
briefing content
POST   /api/briefing/{merchant_id}/send             # Send via WhatsApp +
Soundbox TTS

# ========================================
# INVENTORY
# ========================================
GET    /api/inventory/{merchant_id}                # Inferred stock levels
+ predictions
GET    /api/inventory/{merchant_id}/reorder         # Reorder alerts
GET    /api/inventory/{merchant_id}/deadstock       # Slow-moving items

# ========================================
# DEMO HELPERS
# ========================================
POST   /api/demo/reset                             # Reset to clean seed
data
```

```
POST   /api/demo/simulate-payment                # Fake incoming Paytm QR
payment
POST   /api/demo/simulate-collection             # Fake debtor paying via
link
POST   /api/demo/trigger-alert                   # Trigger cash crunch
alert
POST   /api/demo/advance-time                    # Jump to next day (for
forecast demo)
```

PHASE 5: Synthetic Data Generation

Full 6-Month Merchant Dataset

```
# services/ml-training/generate_synthetic_data.py

"""
Generates realistic 6-month Paytm merchant data for "Sunita Saree Shop, V
Revenue: Rs 3-4L/month | 3 employees | 200+ customers | Paytm QR + Soundb

This dataset must be realistic enough that:
- TFT forecasting shows meaningful patterns
- TabNet PayScore produces interpretable feature importance
- TS2Vec churn detection flags real-looking declining customers
- Collection RL has diverse debtor profiles to learn from
- GST classification covers realistic transaction categories
"""

# Transaction distribution:
# - 20-40 transactions/day
# - Amount: Rs 200-25,000 (saree shop range)
# - 60% UPI, 25% cash, 10% card, 5% wallet
# - Weekly pattern: Mon=slow, Sat=peak, Sun=closed
# - Festival spikes: Navratri (+80%), Diwali (+120%), Holi (+40%)
# - January dip (-30%), March GST rush

# Customer profiles:
# - 200+ unique customers over 6 months
# - 30 "champions" (weekly visitors, high AOV)
# - 50 "loyal" (bi-weekly, moderate AOV)
# - 60 "promising" (monthly, growing)
# - 40 "at risk" (declining frequency)
```

```
# - 20 "lost" (no visit in 60+ days) — churn candidates

# Udhari entries:
# - 35 pending receivables
# - 10 likely-to-pay (high digital activity, responded to past reminders)
# - 15 medium (mixed signals)
# - 10 high-risk (no response, no digital footprint)
# - Amounts: Rs 500 to Rs 25,000

# Expenses (voice-logged):
# - Rent: Rs 15,000/month (recurring)
# - Salary: 3 employees x Rs 12,000/month
# - Stock: Rs 1,50,000-2,50,000/month (variable)
# - Electricity: Rs 2,000-2,500/month
# - Transport: Rs 5,000-10,000/month
# - Miscellaneous: Rs 3,000-5,000/month

# GST data:
# - 5 months filed on time
# - 1 month filed late (Rs 2,100 penalty)
# - 1 ITC mismatch (Rs 8,400)
# - Current month: pending (demo opportunity)
```

PHASE 6: Integration & Deployment

6.1 Full Integration Flow

```
                    ┌──────────────────┐
                    │  PAYTM SOUNDBOX  │
                    │  (Browser Mic)   │
                    └──────────────────┘
                             │
                             │ Audio blob (webm)
                             ▼
┌──────────────────────────────────────────────────────────┐
│                    NEXT.JS FRONTEND                      │
│                                                          │
│  VoiceInput.tsx → POST /api/voice/process                │
│  useSocket.ts → WebSocket connection                     │
│  useDashboardState.ts → Reactive state                   │
│                                                          │
│  Dashboard Page:                                         │
```

```
│  ┌──────────┬──────────┬──────────┬──────────┐    │
│  │LivePnL   │PayScore  │CashFlow  │Activity  │    │
│  │Card      │Gauge     │Mini      │Feed      │    │
│  ├──────────┼──────────┼──────────┼──────────┤    │
│  │Udhari    │CustomerPulse │GSTBadge│Alerts   │    │
│  │Tracker   │              │        │         │    │
│  └──────────┴──────────────┴────────┴─────────┘    │
└──────────────┬──────────────────────────────────────┘
               │ HTTP + WebSocket
               ▼

┌──────────────────────────────────────────────────────┐
│                FASTAPI BACKEND                       │
│                                                      │
│  POST /api/voice/process:                            │
│   1. noisereduce → clean audio                       │
│   2. IndicWhisper → transcript                       │
│   3. IndicBERTv2 → intent + confidence               │
│   4. spaCy NER → entities (amount, person, etc.)     │
│   5. ActionRouter → DB write                         │
│   6. Socket.IO → emit dashboard_update               │
│   7. TTS → generate voice confirmation               │
│   8. Return {transcript, intent, response, delta}    │
│                                                      │
│  LangGraph Master Agent:                             │
│   ├── Collection Agent (RL)                          │
│   ├── CashFlow Agent (TFT + Chronos)                 │
│   ├── PayScore Agent (TabNet + HAN)                  │
│   ├── GST Agent (IndicBERTv2 HSN)                    │
│   ├── Customer Agent (TS2Vec)                        │
│   └── Scheme Agent (CRAG + BGE-M3)                   │
│                                                      │
│  Socket.IO Server → Broadcast events                 │
└──────────────┬──────────────────────────────────────┘
               │
     ┌─────────┴─────────┬─────────┬─────────┐
     ▼                   ▼         ▼         ▼

┌────────┐   ┌────────┐  ┌────────┐  ┌────────┐
│Supabase│   │ Redis  │  │ChromaDB│  │ Groq   │
│Postgres│   │Upstash │  │Vectors │  │  LPU   │
└────────┘   └────────┘  └────────┘  └────────┘
```

6.2 Deployment Architecture

Service	Platform	Plan	Notes
Next.js Frontend	Vercel	Free	Edge network, instant deploys
FastAPI Backend	Railway	Hobby ($5/mo)	Docker, auto-deploy from GitHub
Supabase (Postgres)	Supabase	Free	500MB storage, 2 projects
Redis	Upstash	Free	10K commands/day, serverless
ChromaDB	Embedded in Backend	—	Runs inside FastAPI container
Groq API	Groq Cloud	Free	Rate limits apply, cache responses
WhatsApp API	Meta Cloud API	Free (test)	1000 free conversations/month
Sarvam TTS	Sarvam AI API	Free tier	Hindi voice synthesis
ML Models	Backend container	—	ONNX-exported models loaded at startup

---

PHASE 7: Demo & Presentation Strategy

The 5-Minute Demo Script

Setup: Dashboard on screen, WhatsApp tab ready, teammate's phone visible.

Act 1 — Morning Briefing (60s):

- Show WhatsApp: MunimAI sent morning briefing at 9 AM
- "Kal Rs 34,500 ki sale hui. 3 udhari due. GSTR-3B 20 April ko."
- Switch to dashboard — everything alive with data

Act 2 — Voice Commands / Live P&L (120s) — THE WOW:

1. Speak: "Muneem, Rs 5,000 rent diya" → Dashboard: expense jumps (red flash), margin shifts yellow
2. Simulate Paytm QR: Rs 2,500 arrives → Income increases (green flash)
3. Speak: "Muneem, Rs 45,000 stock kharida" → Profit goes NEGATIVE → AlertBanner slides in

Act 3 — Autonomous Collection (60s):

4. Click "Haan, bhej do" → WhatsApp shows 3 Hindi collection messages with Paytm links

5. Teammate "pays" Rs 8,000 → Udhari turns green, income recovers

Act 4 — Intelligence Layer (60s):
6. Speak: "Muneem, aaj kaisa raha?" → Hindi TTS summary
7. Flash: Forecast page (crisis zone), PayScore (74 → path to 80), GST (ready to file), Schemes (MUDRA eligible)

Act 5 — The Pitch (60s):
"Every MunimAI action = Paytm revenue. Collections → Paytm link. Credit → Paytm loan. The more MunimAI works, the stickier the Paytm ecosystem."

Verification Plan

How to test end-to-end:

1. **Voice Pipeline:** Record 10 Hindi commands → verify STT accuracy, intent classification, entity extraction
2. **Live Dashboard:** Speak command → verify WebSocket event → verify Framer Motion animation < 700ms
3. **Udhari Flow:** Create udhari via voice → verify DB entry → trigger reminder → verify WhatsApp message → simulate payment → verify settlement
4. **Forecast:** Load 6-month data → train TFT → verify 30-day forecast has festival markers + crisis zones
5. **PayScore:** Calculate score for demo merchant → verify 47 features → verify Hindi tips make sense
6. **Churn:** Pre-train TS2Vec → classify demo customers → verify "at risk" customers are flagged correctly
7. **Collection RL:** Run 50 simulated collection rounds → verify Thompson Sampling explores all strategies → verify reward learning
8. **GST:** Auto-classify 100 transactions → verify HSN code accuracy > 90%
9. **Scheme Matching:** Query with Sunita's profile → verify MUDRA, PMEGP matches
10. **Full Demo Run:** Execute demo script 3 times end-to-end without crashes

Critical Files to Modify:

- `apps/web/src/app/page.tsx` — Dashboard composition
- `apps/web/src/components/dashboard/LivePnLCard.tsx` — Hero component
- `apps/web/src/hooks/useSocket.ts` — WebSocket connection
- `apps/web/src/hooks/useDashboardState.ts` — Reactive state management
- `services/ai-engine/main.py` — FastAPI app entry

- `services/ai-engine/services/nlu/pipeline.py` — Core NLU orchestration
- `services/ai-engine/services/agents/master_agent.py` — LangGraph orchestration
- `services/ai-engine/services/action_router.py` — Intent → DB → WebSocket
- `services/ai-engine/services/ml/tft_forecaster.py` — Cash flow prediction
- `services/ai-engine/services/ml/tabnet_scorer.py` — PayScore engine
- `services/ai-engine/services/ml/ts2vec_embedder.py` — Churn detection
- `services/ai-engine/services/agents/collection_agent.py` — RL collection
- `services/ml-training/generate_synthetic_data.py` — Demo data
- `database/schema.sql` — Full database schema